DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-2

ASSIGNMENT 2 – CO2

CO2 Coverage: N-grams Models of Syntax; Counting Words; Unsmoothed N-grams; Smoothing — Backoff, Deleted Interpolation; Entropy; English Word Classes; Tagsets for English; Part of Speech Tagging; Rule-Based Part of Speech Tagging; Stochastic Part of Speech Tagging; Transformation-Based Tagging

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: ***05/11/2026***

Assignment Title: Build and Evaluate an N-gram Language Model and a Part-of-Speech Tagger for a Public Transport Enquiry Helpline (Total Marks – 50)

Duration: 1 week

DSA03 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 2 (CO2)

Assignment Title: Build and Evaluate an N-gram Language Model and a Part-of-Speech Tagger for a Public Transport Enquiry Helpline

Name: Udhayakumar

Register Number: 192424332

Course: Natural Language Processing

Total Marks: 50

Introduction

A public transport enquiry helpline receives thousands of calls every day regarding bus timings, train routes, ticket bookings, fares, cancellations and refunds. The speech recognition system converts callers' speech into text. However, it may produce multiple possible transcripts for the same sentence.

For example:

1. Two tickets to Vellore.
2. To tickets to Vellore.
3. Two tickets to Velore.

An N-gram language model helps identify the most suitable sentence by calculating word sequence probabilities. After selecting the transcript, a Part-of-Speech (POS) tagger identifies the grammatical role of each word.

This assignment explains how N-gram models and three POS-tagging methods can be developed and evaluated for a transport enquiry helpline.

Note about experimental results: The assignment requires an actual corpus containing at least 20,000 utterances, annotated words and speech-recognition hypotheses. These datasets were not provided. Therefore, the answers below explain the methods and calculations, but actual corpus measurements, accuracy values and experimental comparisons must be obtained by running the experiments. Illustrative examples are identified clearly rather than presented as real measurements.

TASK 1 – Counting Words and Building the Vocabulary (3 Marks)

1.1 Types, Tokens, Lemmas and Word Forms

Word counting is an important step in Natural Language Processing because language models depend on word frequencies.

- Token: Every occurrence of a word in a sentence.
- Type: A unique word appearing in the corpus.
- Lemma: The basic dictionary form of a word.
- Word Form: The actual form of a word used in a sentence.

Example:

Sentence: "The buses are running to Chennai."

Tokens: The, buses, are, running, to, Chennai.

Types: The, buses, are, running, to, Chennai.

Lemmas: the, bus, be, run, to, Chennai.

1.2 Tokenisation Decisions

Tokenisation is the process of dividing sentences into individual words.

The following decisions are used:

1. Case: Convert all words into lowercase. Therefore, Chennai and chennai are treated as the same word.
2. Punctuation: Remove unnecessary punctuation marks.
3. Numerals: Retain numbers as individual tokens.
4. Disfluencies: Retain spoken words such as "uh" and "um" when they are part of the transcript.
5. Sentence boundaries: Use <s> and </s> to identify the beginning and ending of sentences.

For example, the time 6:30 can be treated as one token or separated into 6 and 30. Treating it as one token preserves the meaning of the time expression.

1.3 Zipf's Law

Zipf's Law states that the frequency of a word is approximately inversely proportional to its rank.

$$f(r) \approx \frac{C}{r}$$

Where:

- $f(r)$ = Frequency of the word.
- r = Rank of the word.
- C = Constant.

In a transport enquiry corpus, common words such as "the", "to", "bus" and "ticket" appear frequently. Station names and special fare terms occur less frequently.

A log-log rank-frequency graph can be used to examine whether the corpus follows Zipf's Law. Domain-specific words may deviate because transport enquiries repeatedly use words such as bus, train and ticket.

1.4 Vocabulary and Unknown Words

Vocabulary contains all unique words identified in the training corpus.

Rare words can be replaced with <UNK> (unknown word) to handle words that do not appear during training.

Frequency Threshold	Treatment
1	Retain words occurring at least once
2	Replace words occurring once with <UNK>
3	Replace words occurring fewer than three times
5	Replace words occurring fewer than five times

The vocabulary size decreases when the threshold increases.

1.5 Out-of-Vocabulary Rate

The OOV rate identifies how many development-set words are missing from the training vocabulary.

$$OOV = \frac{Unseenwords}{Totaldevelopmentwords} \times 100$$

A high OOV rate reduces language-model accuracy because the model cannot estimate reliable probabilities for unknown words.

Result: Word counting and vocabulary construction provide the foundation for building accurate N-gram language models.

TASK 2 – Unsmoothed N-gram Models (4 Marks)

2.1 Chain Rule of Probability

The chain rule calculates the probability of a complete word sequence by multiplying the conditional probabilities of individual words.

$$P(w_1, w_2, ..., w_n) = \prod_{i=1}^{n} P(w_i | w_1, ..., w_{i-1})$$

However, calculating the complete history of every word requires a large amount of data. Therefore, the Markov assumption is used.

2.2 Types of N-gram Models

1. Unigram Model

A unigram model predicts a word without considering the previous words.

$$P(w) = \frac{C(w)}{N}$$

2. Bigram Model

A bigram model predicts a word based on the immediately previous word.

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

3. Trigram Model

A trigram model predicts a word based on the previous two words.

$$P(w_i | w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i)}{C(w_{i-2}, w_{i-1})}$$

2.3 Example Probability Calculation

Consider the sentence:

"Book a bus ticket."

Using the bigram model:

\[\begin{aligned} P(S)=&P(book|<s>)\\ &\times P(a|book)\\ &\times P(bus|a)\\ &\times P(ticket|bus)\\ &\times P(</s>|ticket) \end{aligned} \]

Each probability is calculated using the corresponding bigram frequency from the training corpus.

If a bigram does not occur in the training data, its probability becomes zero.

For example, the sentence "Please reserve a seat" may receive zero probability if the bigram "reserve a" is missing from the training corpus.

2.4 Sparsity Problem

Sparsity occurs when many possible word combinations are not present in the training data.

$$Sparsity = \frac{UnobservedN-grams}{TotalpossibleN-grams}$$

Unigram models have fewer context requirements. Bigram models provide more contextual information, while trigram models provide better local context but require more training data.

2.5 Sentence Generation

The models can also generate sentences.

- Unigram: Produces words independently and may generate meaningless sentences.
- Bigram: Produces sentences using the previous word.
- Trigram: Uses the previous two words and generally produces better local word combinations.

Result: Unsmoothed N-gram models estimate word sequence probabilities using frequency counts, but they suffer from zero probabilities and data sparsity.

TASK 3 – Smoothing and Discounting (5 Marks)

3.1 Need for Smoothing

In an unsmoothed model, an unseen word combination receives zero probability. This does not mean the combination is impossible in English. It only means that it was not found in the training corpus.

Smoothing solves this problem by assigning a small probability to unseen combinations.

A valid probability distribution must satisfy:

$$\sum_{w} P(w|h) = 1$$

3.2 Add-One (Laplace) Smoothing

Add-one smoothing adds one to every possible word count.

$$P(w|h) = \frac{C(h,w)+1}{C(h)+V}$$

Where:

- $C(h, w)$ = Count of the word combination.
- $C(h)$ = Count of the context.
- V = Vocabulary size.

It prevents zero probabilities, but it may assign too much probability to unseen words when the vocabulary is large.

3.3 Add-k Smoothing

Add-k smoothing is similar to Laplace smoothing, but it adds a smaller value k.

$$P(w|h) = \frac{C(h,w)+k}{C(h)+kV}$$

Here, k is generally between 0 and 1.

It provides more control over the amount of smoothing and can be tuned using development data.

3.4 Good–Turing Discounting

Good–Turing discounting adjusts observed counts to estimate the probability of unseen events.

$$C^* = (r + 1)\frac{N_{r+1}}{N_r}$$

Where:

- r = Original frequency.
- N_r = Number of N-gram types occurring exactly r times.
- N_{r+1} = Number of N-gram types occurring $r + 1$ times.

The probability mass assigned to unseen events is estimated using:

$$P_{unseen} \approx \frac{N_1}{N}$$

Here, N_1 represents the number of N-gram types occurring once.

3.5 Comparison of Smoothing Methods

Method	Advantages	Limitations
Add-One	Simple and prevents zero probability	Over-smoothing
Add-k	Flexible smoothing	Requires choosing k
Good–Turing	Estimates unseen probability mass	Needs reliable frequency counts

Result: Smoothing techniques improve N-gram models by assigning probability to unseen word combinations. Good–Turing or a discounted model with backoff is generally preferable for a larger vocabulary.

TASK 4 – Backoff and Deleted Interpolation

4.1 Backoff

Backoff is a language-model technique that uses a lower-order N-gram when a higher-order N-gram is unavailable.

For example, if a trigram is not found, the model checks the corresponding bigram. If the bigram is also unavailable, it uses the unigram.

Katz Backoff: Katz backoff uses discounting to reduce the probability of observed N-grams. It uses a normalisation factor called alpha (α) to distribute the remaining probability mass correctly.

4.2 Stupid Backoff

Stupid backoff uses a fixed discount factor when a higher-order N-gram is missing.

For example:

$$Score = 0.4 \times P(lowerorder)$$

It is useful for ranking possible sentences. However, its output is a score rather than a properly normalised probability.

4.3 Linear Interpolation

Linear interpolation combines unigram, bigram and trigram probabilities.

\[\begin{aligned} P(w_i)=&\lambda_1P(w_i)\\ &+\lambda_2P(w_i|w_{i-1})\\ &+\lambda_3P(w_i|w_{i-2},w_{i-1}) \end{aligned} \]

The weights must satisfy:

$$\lambda_1 + \lambda_2 + \lambda_3 = 1$$

4.4 Deleted Interpolation

Deleted interpolation estimates the best weights by temporarily holding out part of the training data.

It evaluates how well unigram, bigram and trigram models predict the held-out words. The weights are selected based on their predictive performance.

Example of an illustrative weight allocation:

Context Count	Unigram	Bigram	Trigram
Low (0–2)	0.55	0.30	0.15
Medium (3–10)	0.25	0.45	0.30
High (above 10)	0.10	0.30	0.60

These are example weights only. Actual weights must be calculated using the training corpus.

4.5 Comparison

Backoff uses a lower-order model only when the higher-order model fails. Interpolation combines all available N-gram orders.

Interpolation is useful when different context lengths provide useful information. Backoff is useful when higher-order counts are reliable but some combinations are missing.

Result: Backoff and deleted interpolation improve prediction by combining information from different N-gram orders.

TASK 5 – Entropy, Cross-Entropy and Perplexity (4 Marks)

5.1 Entropy

Entropy measures the uncertainty present in a probability distribution.

$$H(P) = -\sum_{x} P(x)\log_2 P(x)$$

A higher entropy value indicates greater uncertainty, while a lower value indicates that the outcomes are more predictable.

5.2 Cross-Entropy

Cross-entropy measures how well a probability model predicts actual data.

$$H(P, Q) = -\sum_{x} P(x)\log_2 Q(x)$$

A lower cross-entropy indicates better prediction performance.

5.3 Perplexity

Perplexity measures the uncertainty of a language model.

$$PP = 2^{H}$$

A lower perplexity means the model predicts the next word more effectively.

For example, a perplexity of 90 means the model has an effective uncertainty similar to choosing among 90 equally likely alternatives.

5.4 Example Entropy Calculation

Consider four possible outcomes.

Nearly uniform distribution:

$$P = (0.25, 0.25, 0.25, 0.25)$$

$$H = -4(0.25\log_2 0.25) = 2\ bits$$

Sharply peaked distribution:

$$P = (0.85, 0.05, 0.05, 0.05)$$

$$H = -\sum P(x)\log_2 P(x)$$

$$H \approx 0.848\ bits$$

5.5 Model Comparison

The following table shows the format for comparing models. The numbers are illustrative, not measurements from the required corpus.

Model	Illustrative Perplexity
Unigram	38.5
Bigram with Laplace	24.2
Interpolated Trigram	19.8
Katz Backoff	18.9

Katz backoff has the lowest illustrative perplexity in this example.

Perplexity must be compared using the same test dataset, vocabulary and <UNK> treatment.

5.6 Word Error Rate

Word Error Rate (WER) evaluates speech-recognition performance.

$$WER = \frac{S+D+I}{N} \times 100$$

Where:

- S = Substitutions.
- D = Deletions.
- I = Insertions.
- N = Number of words in the reference transcript.

The language model can rescore five competing recognition hypotheses and select the best one. WER is then calculated before and after rescoring.

Result: Entropy, cross-entropy and perplexity help evaluate language-model performance, while WER measures the quality of the selected speech transcript.

TASK 6 – English Word Classes and Tagset Design (4 Marks)

6.1 Open and Closed Word Classes

English words are divided into two major categories.

Open Word Classes:

These classes can accept new words.

- Nouns
- Verbs
- Adjectives
- Adverbs

Closed Word Classes:

These classes contain a limited set of words.

- Pronouns
- Prepositions
- Determiners
- Conjunctions
- Auxiliary verbs
- Particles

Closed-class words are important because they frequently occur in sentences and help identify grammatical relationships.

6.2 Ambiguous Words

Some English words can have different grammatical roles depending on their usage.

Word	Noun Usage	Verb Usage
Book	Read a book	Book a ticket
Return	Return ticket	Return tomorrow
Stand	Bus stand	Please stand
Train	The train arrived	Train the staff

6.3 Proposed Domain-Specific Tagset

The following tagset is designed for transport enquiry sentences.

Tag	Meaning	Penn Treebank Mapping
NOUN	Common noun	NN, NNS
PROPN	Proper noun	NNP, NNPS
VERB	Base verb	VB
VERB3	Third-person verb	VBZ
VERBP	Past-tense verb	VBD
VERBG	Present participle	VBG
VERBN	Past participle	VBN
ADJ	Adjective	JJ, JJR, JJS
ADV	Adverb	RB, RBR, RBS
DET	Determiner	DT, PDT
PRON	Pronoun	PRP, PRP$
ADP	Preposition	IN
CONJ	Conjunction	CC
AUX	Auxiliary	MD and auxiliary forms
NUM	Number	CD

PART	Particle	RP, TO
INTJ	Interjection	UH
WH	Wh-word	WDT, WP, WRB
PUNCT	Punctuation	Punctuation tags
UNK	Unknown word	No direct equivalent

6.4 Annotation and Agreement

At least 3,000 tokens must be manually tagged. Among these, 500 tokens must be independently tagged by two annotators.

Agreement is calculated using:

$$Agreement = \frac{Matchingtags}{Totaltags} \times 100$$

Cohen's Kappa measures agreement after accounting for chance agreement.

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

Where:

- P_o = Observed agreement.
- P_e = Expected agreement.

Result: A domain-specific tagset helps identify grammatical roles and supports accurate extraction of transport information.

TASK 7 – Rule-Based Part-of-Speech Tagging (4 Marks)

7.1 Architecture

A rule-based POS tagger works using two main stages.

Stage 1 – Dictionary

A dictionary stores words and their possible grammatical tags.

Example:

- Book – NOUN / VERB
- Train – NOUN / VERB
- Return – NOUN / VERB
- Tickets – NOUN

Stage 2 – Context Rules

Rules examine surrounding words and remove incorrect tags.

7.2 Disambiguation Rules

The following 15 rules can be used for the transport enquiry domain.

1. If a word follows a determiner, prefer NOUN or ADJ.
2. If a word follows "to", prefer a base-form VERB when the context indicates an action.
3. If a word follows a pronoun, prefer VERB when grammatically suitable.
4. If a word ends in "-ly", prefer ADV.
5. If a word ends in "-ing" after an auxiliary verb, prefer VERBG.
6. If a word ends in "-ed", consider VERBP or VERN.
7. If a word is a known station name, assign PROPN.
8. If a word contains only digits, assign NUM.
9. If a word is a known preposition, assign ADP.
10. If a word occurs between a determiner and a noun, consider ADJ.
11. If a word follows an auxiliary verb, consider a verb form.
12. If a word is "and", "but" or "or", assign CONJ.
13. If a possessive word appears before a noun, assign PRON.
14. If a word is "please" at the beginning of a request, assign INTJ according to the annotation policy.
15. If a word is followed by a destination or route expression, use the surrounding context to distinguish a noun from a verb.

7.3 Evaluation

The rule-based tagger is evaluated using:

$$Accuracy = \frac{Correctlytaggedwords}{Totalwords} \times 100$$

Ambiguous-token accuracy should also be measured separately because these words are more difficult to classify.

7.4 Common Errors

Common errors include:

- Noun and verb confusion.
- Adjective and adverb confusion.
- Incorrect verb forms.
- Unknown station names.
- Code-mixed words.

If two rules conflict, the more specific rule should be applied before the general rule.

Result: Rule-based tagging uses dictionaries and linguistic rules to identify grammatical roles. It is understandable and easy to modify, but maintaining many rules can become difficult.

TASK 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm (5 Marks)

8.1 Hidden Markov Model

A Hidden Markov Model (HMM) is a statistical method used to identify the most likely sequence of POS tags for a sentence.

It uses two important probabilities:

1. Transition Probability: Probability of one tag following another.
2. Emission Probability: Probability of a word being generated by a particular tag.

The objective is to find the most probable tag sequence for the given words.

$$T^* = \arg\max_T P(T|W)$$

Using Bayes' rule:

$$P(T|W) \propto P(W|T)P(T)$$

8.2 Transition Probability

Transition probability calculates the probability of a tag based on the previous tag.

$$P(t_i|t_{i-1}) = \frac{C(t_{i-1},t_i)}{C(t_{i-1})}$$

8.3 Emission Probability

Emission probability calculates the probability of a word being associated with a particular tag.

$$P(w_i|t_i) = \frac{C(t_i,w_i)}{C(t_i)}$$

Smoothing can be applied to avoid zero probabilities.

8.4 Viterbi Algorithm

The Viterbi algorithm identifies the most probable tag sequence without checking every possible combination.

Steps:

1. Initialise the probability of each possible first tag.
2. Calculate transition and emission probabilities.
3. Find the highest probability path for each tag.
4. Store the previous tag using a back-pointer.
5. Continue until the final word.
6. Select the highest probability final tag.
7. Follow the back-pointers to obtain the complete tag sequence.

The recurrence is:

$$V_i(t) = \max_s [V_{i-1}(s)P(t|s)P(w_i|t)]$$

8.5 Example

Sentence:

"Please book two tickets to Vellore."

Word	Expected Tag
Please	INTJ
Book	VERB
Two	NUM
Tickets	NOUN
To	ADP
Vellore	PROPN

The Viterbi algorithm calculates the most probable sequence using the transition and emission probabilities.

8.6 Comparison

- Most Frequent Tag: Assigns the most common tag to each word.
- Viterbi: Uses probabilities and sentence context.
- Exhaustive Search: Checks every possible tag sequence, which becomes computationally expensive.

If there are m possible tags and n words, Viterbi requires approximately:

$$O(nm^2)$$

Exhaustive search requires approximately:

$$O(m^n)$$

8.7 Unknown Word Handling

Unknown words can be identified using:

- Suffix patterns.
- Capitalisation.
- Open-class word probabilities.
- Station-name dictionaries.

For example, a capitalised unknown word following "to" may be a destination name and can be assigned PROPN.

Result: HMM tagging uses statistical probabilities, while the Viterbi algorithm efficiently finds the most probable POS-tag sequence.

TASK 9 – Transformation-Based (Brill) Tagging (4 Marks)

9.1 Introduction

Transformation-based tagging is a method that improves POS tagging by learning correction rules from previous tagging errors.

It begins with an initial tagger and repeatedly applies rules to correct incorrect tags.

9.2 Working Procedure

1. The initial tagger assigns tags using the most frequent tag for each word.
2. The system compares predicted tags with correct tags.
3. It identifies the errors.
4. Candidate transformation rules are generated.
5. The rule that corrects the most errors is selected.
6. The selected rule is applied to the training data.
7. The process continues until improvement becomes small.

9.3 Rule Templates

The following six templates can be used:

1. Change the current tag based on the previous tag.
2. Change the current tag based on the next tag.
3. Change the current tag based on the previous word.
4. Change the current tag based on the next word.
5. Change the current tag based on the previous two tags.
6. Change the current tag based on the word's suffix.

9.4 Example Transformation Rules

Rule 1: Change NOUN to VERB when the previous word is "to".

Example: "to book a ticket".

Rule 2: Change a possible VERB to NOUN when it follows a determiner and appears before another noun.

Example: "the train station".

These rules help improve grammatical classification.

9.5 Evaluation and Overfitting

The system is evaluated using tagging accuracy after applying different numbers of transformation rules.

Initially, accuracy increases because the rules correct common errors. After some point, additional rules may start correcting only training-specific cases. This is called overfitting.

A graph of accuracy against the number of transformations can identify the point where performance stops improving.

9.6 Advantages

- Learns rules from training examples.
- Produces understandable correction rules.
- Can improve a basic tagger.
- Combines statistical learning with linguistic information.

Result: Brill tagging improves initial POS tagging through learned transformation rules. It provides understandable corrections but may overfit if too many rules are applied.

TASK 10 – Implement and Test the Models (3 Marks)

10.1 Implementation

The complete system can be implemented using Python and the NLTK library.

The main implementation stages are:

1. Read and clean the transport enquiry corpus.
2. Tokenise the sentences.
3. Create training, development and testing datasets.
4. Calculate unigram, bigram and trigram frequencies.
5. Implement smoothing and discounting.
6. Implement backoff and interpolation.
7. Calculate entropy and perplexity.
8. Create the POS tag dictionary.
9. Implement rule-based tagging.
10. Implement HMM tagging and the Viterbi algorithm.
11. Implement Brill tagging.
12. Evaluate all models.

10.2 Python Program – N-gram Model

from collections import Counterimport mathdef ngram_counts(sentences, n): counts = Counter() contexts = Counter() for sentence in sentences: words = ["<s>"] * (n - 1) words += sentence.lower().split() words += ["</s>"] for i in range(n - 1, len(words)): gram = tuple(words[i-n+1:i+1]) counts[gram] += 1 contexts[gram[:-1]] += 1 return counts, contextsdef add_k_probability(word, context, counts, contexts, vocab_size, k=0.1): numerator = counts[tuple(context) + (word,)] + k denominator = contexts[tuple(context)] + k * vocab_size return numerator / denominatorsentences = ["book a bus ticket", "what is the bus fare", "train to vellore", "book two tickets"]

10.3 Testing

The following test cases should be performed:

Test Case	Expected Result
Unseen bigram	Unsmoothed probability should be zero
Smoothing	Unseen events should receive non-zero probability
Interpolation	Weights should sum to one
Viterbi	Best tag sequence should be recovered
Brill tagging	Transformation rules should correct tagging errors

10.4 Performance Evaluation

The following metrics should be recorded for each model:

- Training time.
- Model memory size.
- Perplexity.
- POS tagging accuracy.
- Processing speed in tokens per second.

Two useful graphs are:

1. Perplexity versus N-gram order.
2. Tagging accuracy versus training-set size.

Result: Python and NLTK can be used to implement and test N-gram language models and POS-tagging methods using a reproducible experimental procedure.

TASK 11 – Compare, Recommend and Defend the Design (2 Marks)

11.1 Language Model Comparison

Model	Advantage	Limitation
Unigram	Simple and fast	Does not consider context
Bigram	Considers previous word	Limited context
Trigram	Considers previous two words	Requires more training data
Katz Backoff	Handles unseen combinations through lower-order models	More complex implementation
Interpolation	Combines different N-gram orders	Requires suitable weights

11.2 POS Tagger Comparison

Tagger	Advantage	Limitation
Rule-Based	Easy to understand	Requires manual rules
HMM	Uses statistical probabilities	Depends on annotated data
Brill	Learns correction rules	Can overfit

11.3 Recommendation

For the original English transport enquiry system, an interpolated trigram or Katz backoff language model is a suitable choice because it uses word context while handling unseen combinations.

For POS tagging, an HMM with the Viterbi algorithm is a practical choice because it considers the complete tag sequence and uses statistical probabilities.

The final selection must be confirmed using measured perplexity, WER, tagging accuracy, memory and processing speed.

Result: A discounted N-gram model with backoff or interpolation and an HMM tagger provide a practical starting design for the transport enquiry helpline.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN (8 Marks)

A. Impact Analysis (2 Marks)

12.1 Changes in Requirements

The transport enquiry helpline is expanded to include metro and intercity rail services. It now receives new station names, train names and code-mixed sentences.

Example:

"Chennai to Madurai bus timing enna."

The original system was mainly designed for English sentences. The new requirement introduces several problems:

1. The vocabulary increases because of new station and train names.
2. More than 15% of words are not present in the training vocabulary.
3. The original tagset does not support code-mixed words.
4. Existing transition probabilities may not represent the new language patterns.
5. Speech recognition may produce inconsistent spellings of station names.

12.2 Impact of Unknown Words

Unknown words are especially important because many of them are station names and destinations.

If the system cannot recognise a destination, it may select the wrong route or provide incorrect travel information.

A high OOV rate can increase perplexity and reduce POS tagging accuracy.

Result: The expanded requirement reduces the reliability of the original language model and POS tagger.

B. Evaluation of the Original Design (2 Marks)

12.3 Testing New Utterances

Consider the following three examples.

Sentence	Problem	Expected Failure
Book a ticket to Poonamallee Metro	Unseen station name	Unknown-word prediction
Chennai to Madurai bus timing enna	Code-mixed sentence	English-only tagset
Metro from Koyambedu to Alandur eppo	Station names and Tamil word	OOV and tagging errors

12.4 Model Degradation

The original model may experience:

- Increased perplexity because of unknown words.
- Increased WER because of inconsistent station-name spellings.
- Reduced POS accuracy because the tagger cannot classify code-mixed words.
- Incorrect destination identification.

The degradation must be measured by testing the original system against a newly collected and manually annotated dataset.

12.5 Components That Need Improvement

The language model may degrade quickly because of the large number of unseen words. The POS tagger also suffers because its original tagset does not support code-mixed words.

A station-name gazetteer, character-level model and extended tagset can reduce these problems.

Result: The original system requires additional vocabulary and language support to handle metro, rail and code-mixed enquiries.

C. Adaptation or Redesign (2 Marks)

12.6 Recommended Decision – MODIFY

The best decision is to modify the original system rather than completely replace it.

The following improvements are recommended.

1. Add a Station-Name Gazetteer

Create a dictionary containing station names, train names and major destinations.

Example:

- Chennai Central
- Koyambedu
- Alandur
- Madurai
- Vellore

This helps the system identify important transport-related proper nouns.

2. Add Character N-gram Support

Character N-gram models identify spelling patterns within words. They can help recognise spelling variations and unseen words.

Example:

- Vellore
- Velore
- Vellor

3. Extend the POS Tagset

Introduce a new tag for code-mixed words.

4. Retrain the Language Model

New transport enquiries should be added to the training dataset. The N-gram probabilities and interpolation weights should be recalculated.

5. Improve Unknown-Word Handling

Use suffix patterns, capitalisation, station dictionaries and character N-gram models to handle unseen words.

12.7 Model Refreshing

The vocabulary and models should be updated regularly because new transport services and stations are introduced.

A monthly refresh is a reasonable initial plan. The schedule can be adjusted according to changes in vocabulary and model performance.

Result: Modifying the existing system with a station gazetteer, code-mixed tagset and improved unknown-word handling provides better support for the expanded transport enquiry system.

D. Evidence-Based Defence (2 Marks)

12.8 Final Design Decision

The modified system is recommended because it maintains the useful parts of the original design while solving the new requirements.

The N-gram model continues to provide word sequence probabilities. The HMM tagger identifies grammatical roles. Additional station dictionaries and character-level models help handle unseen names and spelling variations.

Advantages

1. Supports new station and train names.
2. Improves unknown-word handling.
3. Supports code-mixed sentences.
4. Reduces errors caused by spelling variations.
5. Reuses the existing language-model framework.

Conclusion

This assignment explains the development of an N-gram language model and three POS-tagging methods for a public transport enquiry helpline.

Unigram, bigram and trigram models calculate word sequence probabilities. Smoothing, backoff and interpolation improve prediction by handling unseen word combinations. Entropy and perplexity measure language-model performance.

Rule-based, HMM and Brill taggers identify the grammatical roles of words. Among these methods, HMM tagging with the Viterbi algorithm provides a practical statistical approach.

For the expanded metro and intercity rail requirement, modifying the original system with a station-name gazetteer, character N-gram support and an extended POS tagset is recommended.

The complete system can help improve transcript selection and extract important transport information such as destinations, routes, timings and fares.

References

1. Daniel Jurafsky and James H. Martin, Speech and Language Processing, chapters on N-gram Language Models and Part-of-Speech Tagging.
2. Eric Brill, “Transformation-Based Error-Driven Learning and Natural Language Processing,” Computational Linguistics, 1995.
3. Natural Language Toolkit (NLTK), Official Documentation: https://www.nltk.org/
4. Church, K. W. and Gale, W. A., research on Good–Turing and deleted estimation methods for language modelling.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Counting Words and Building the Vocabulary	3
Task 2 – Unsmoothed N-gram Models	4
Task 3 – Smoothing and Discounting	5
Task 4 – Backoff and Deleted Interpolation	4
Task 5 – Entropy, Cross-Entropy and Perplexity	4
Task 6 – English Word Classes and Tagset Design	4
Task 7 – Rule-Based Part-of-Speech Tagging	4
Task 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm	5
Task 9 – Transformation-Based (Brill) Tagging	4
Task 10 – Implement and Test the Models	3
Task 11 – Compare, Recommend and Defend the Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**